IPFS Gateway Migration Proof of Concept

Content-Addressed Storage with QUIC Server-Side Migration

PoC Documentation • July 2026

This document describes a proof-of-concept demonstrating QUIC server-side connection migration between two IPFS gateways. Both gateways pin the same content (identified by CID), and the primary gateway advertises the preferred gateway's address during the QUIC handshake. IPFS's content-addressing model guarantees that content served by either gateway is cryptographically identical, making IPFS a natural fit for migration scenarios. The PoC runs on our existing four-machine testbed using modified Neqo binaries.

1. Overview

IPFS (InterPlanetary File System) is a peer-to-peer content-addressed storage network. Since Kubo v0.18+ (released 2023), QUIC is the default transport for IPFS peer-to-peer communication, replacing the older TCP+Noise stack. This makes IPFS nodes natural candidates for QUIC server-side migration experiments.

IPFS gateways bridge the gap between the IPFS network and regular HTTP clients. They serve IPFS content over standard HTTP/HTTPS (and increasingly HTTP/3) to browsers and other clients that do not run an IPFS node. Production gateways like ipfs.io and dweb.link already operate as multi-backend clusters.

This PoC demonstrates migration between two IPFS gateways that both pin the same content, identified by its CID (Content Identifier). The key insight is that IPFS's content-addressing model provides a built-in integrity guarantee: the same CID always resolves to the same content, regardless of which gateway serves it.

IPFS uses QUIC as its default transport since Kubo v0.18+ (2023). IPFS gateways serve content over HTTP/3 to regular browsers. This PoC migrates a client between two IPFS gateways pinning the same CID.

2. Architecture

The PoC uses our existing four-machine testbed on the same LAN (141.217.168.x):

Role	Machine	Components
Primary Gateway	opti7040 (.152)	Kubo IPFS daemon + Neqo HTTP/3 server (webseed-primary)
Preferred Gateway	homeserver2 (.143)	Kubo IPFS daemon + preferred-server binary
Client	optiplex7010 (.127)	Firefox browser or neqo-client
Redis (optional)	Proxmox VM (.200)	State transfer backend (Redis KV or Pub/Sub)

Both the primary and preferred gateways run a local Kubo IPFS node, each pinning the same test content. The primary gateway fetches content from its local IPFS HTTP gateway (localhost:8080) and serves it to clients over HTTP/3 via the webseed-primary binary. During the QUIC handshake, the primary advertises the preferred gateway's address (.143) using the preferred_address transport parameter.

3. How It Works

1. Pin content on both nodes: Both IPFS nodes pin the same content, identified by a CID (Content Identifier). The CID is a cryptographic hash of the content.

2. Primary fetches from local IPFS: The primary server's webseed-primary binary fetches content from the local IPFS gateway at localhost:8080/ipfs/<CID>.

3. Serve via HTTP/3: The primary serves the fetched content to the client over HTTP/3 using the modified Neqo stack.

4. Advertise preferred address: During the QUIC handshake, the primary includes preferred_address = 141.217.168.143 in its transport parameters.

5. **Client migrates:** The client validates the path to the preferred server (PATH_CHALLENGE / PATH_RESPONSE) and migrates the connection.

6. **Content integrity guaranteed:** Because both gateways pin the same CID, the content is cryptographically identical. IPFS guarantees this by design -- the CID IS the hash of the content.

Content integrity is GUARANTEED by IPFS: the CID is a cryptographic hash of the content. Same CID = same content, regardless of which gateway serves it.

4. IPFS-Specific Advantages

Content Addressing

In IPFS, content is identified by its CID -- a hash of the content itself. This means that the same CID always refers to the same content, no matter which node serves it. Unlike traditional URLs (which identify a location), CIDs identify the data. This is a fundamental property that makes IPFS uniquely suited for server migration: there is no question of whether the preferred server will serve different content.

Built-in Integrity Verification

Traditional CDNs require external mechanisms (e.g., SHA-256 checksums, SRI hashes) to verify content integrity after migration. With IPFS, integrity is intrinsic. The CID itself is the verification. If the content does not match the CID hash, the IPFS node will reject it. No additional verification layer is needed.

Multiple Providers

The same CID can be provided (pinned) by many IPFS nodes simultaneously. This is a natural fit for migration: any node that pins the CID can serve as a migration target. In production, IPFS gateway clusters like ipfs.io already run multiple backends behind a load balancer -- migration simply formalizes this at the QUIC layer.

Gateway Clusters

Production IPFS gateways (ipfs.io, dweb.link, cf-ipfs.com) already operate as multi-server clusters. QUIC server-side migration provides a protocol-native mechanism for redirecting clients between backend servers without relying on DNS or HTTP redirects. This reduces latency (no extra round-trip) and is transparent to clients.

5. Setup Scripts

Three shell scripts automate the IPFS environment setup and teardown:

setup_ipfs.sh

Installs Kubo IPFS on both server machines (opti7040 and homeserver2), initializes IPFS repositories, starts the IPFS daemons, and pins test content on both nodes. Run from the client machine -- it uses SSH to configure the remote servers.

```
setup_ipfs.sh — Install Kubo, init repos, pin test content on both servers
```

test_ipfs_migration.sh

Runs the end-to-end migration test. Starts the preferred server on homeserver2, starts the primary server on opti7040 (with IPFS content), and verifies that a client can connect, migrate, and receive the correct content (verified by CID).

```
test_ipfs_migration.sh — End-to-end migration test with IPFS content verification
```

teardown_ipfs.sh

Stops the IPFS daemons on both servers and cleans up temporary files. Does not uninstall Kubo or remove the IPFS repositories.

```
teardown_ipfs.sh — Stop IPFS daemons on both servers
```

6. How to Run

Step 1: Setup IPFS on both servers

Run from the client machine (optiplex7010). This installs Kubo, initializes repositories, and pins test content on both gateway servers.

```
./setup_ipfs.sh
```

Step 2: Start the preferred server

On homeserver2 (.143), start the preferred server binary. This listens for incoming migration state on port 9999 and serves HTTP/3 on port 4433.

```
preferred-server 141.217.168.143:4433 9999
```

Step 3: Start the primary server

On opti7040 (.152), start the webseed-primary binary. It fetches content from the local IPFS gateway and serves it over HTTP/3, advertising .143 as the preferred address.

```
webseed-primary 0.0.0.0:4433 141.217.168.143:4433 /tmp/ipfs_serve_content.bin
```

Step 4: Test the migration

Run the test script from the client machine, or open Firefox and navigate to https://141.217.168.152:4433/. The page should load via HTTP/3 with a transparent migration to the preferred server.

```
./test_ipfs_migration.sh
```

7. Scenarios

Gateway Load Balancing

When an IPFS gateway becomes overloaded, it can use QUIC preferred_address to migrate incoming connections to a less-loaded gateway that pins the same content. Unlike DNS-based load balancing, this happens mid-connection without requiring a new TLS handshake, reducing latency and preserving connection state.

Gateway Maintenance

Before shutting down a gateway node for maintenance, existing connections can be gracefully drained to the preferred server. New connections are directed to the preferred server immediately via preferred_address, while existing connections complete their current transfers before migrating. This enables zero-downtime maintenance of gateway infrastructure.

Geographic Optimization

An IPFS gateway can accept initial connections (via anycast or DNS) and then redirect clients to a geographically closer gateway. After the initial handshake reveals the client's IP address, the primary gateway selects the optimal preferred server based on network topology. The client migrates transparently, improving latency for subsequent requests without requiring client-side changes.

8. Limitations

Peer Identity

IPFS nodes have PeerIDs that are cryptographically tied to their TLS certificates. Migration between different peers requires changes to libp2p's authentication layer, since the receiving node has a different identity. This PoC operates at the gateway layer (HTTP/3), where PeerID authentication is not enforced for HTTP clients.

Peer Identity: IPFS PeerIDs are tied to TLS certificates. Migration between different peers requires libp2p authentication changes.

Bitswap Streams

Active Bitswap streams (the protocol IPFS uses for peer-to-peer block exchange) carry complex state including want-lists, block requests, and session information. This state cannot be easily serialized and transferred between nodes. Migration of native IPFS peer-to-peer connections would require significant Bitswap protocol modifications.

Gateway-Mode Only

This PoC operates in gateway mode -- serving IPFS content over HTTP/3 to standard clients (browsers). It does not attempt to migrate native peer-to-peer IPFS transport connections (which use libp2p + QUIC). The gateway-mode approach is practical because it requires no changes to IPFS clients and leverages standard HTTP/3 semantics.

This PoC uses gateway-mode (HTTP/3), not native peer-to-peer IPFS transport. Native IPFS migration would require libp2p and Bitswap protocol changes.

9. Comparison: IPFS vs Traditional CDN

The following table compares IPFS gateway migration with traditional CDN approaches across key dimensions relevant to server-side connection migration:

Feature	Traditional CDN	IPFS Gateway + Migration
Content verification	None (trust server)	CID hash (cryptographic)
Content availability	Operator-managed	P2P replication
Migration safety	Data could differ	Guaranteed identical (same CID)
Client changes	None	None (HTTP/3 standard)

The key advantage of IPFS for migration is **content verification**: with traditional CDNs, there is no protocol-level guarantee that the preferred server will serve the same content as the primary. With IPFS, the CID cryptographically binds the content to its hash. If the preferred gateway returns different data, the CID will not match, and the client (or an intermediate verification layer) can detect the discrepancy.

IPFS + QUIC migration provides cryptographic content integrity guarantees that traditional CDNs cannot match. The CID ensures identical content across all gateways.